



(<https://docs.aws.amazon.com>)



[Get started](#)  
(#)

[Service guides](#)  
(#)

[Developer tools](#)  
(#)



[Documentation](#)

(<https://docs.aws.amazon.com/index.html>)



[AWS Identity and Access Management](#)

(<https://docs.aws.amazon.com/iam/index.html>)



User G...

# Security best practices in IAM

[↓ PDF \(/pdfs/IAM/latest/UserGuide/iam-ug.pdf#best-practices\)](#)

[↓ RSS \(aws-iam-release-notes.rss\)](#)

[↓ Markdown \(best-practices.md\)](#)

[⚡ Agent Setup](#)



Focus mode

## ► On this page

To help secure your AWS resources, follow these best practices for AWS Identity and Access Management (IAM).

After you implement these best practices, see [Security in IAM and AWS STS](#) ([./security.html](#)) for additional security topics including compliance validation, resilience, and infrastructure security.

## Topics

- [Require human users to use federation with an identity provider to access AWS using temporary credentials \(#bp-users-federation-idp\)](#)
- [Require workloads to use temporary credentials with IAM roles to access AWS \(#bp-workloads-use-roles\)](#)
- [Require multi-factor authentication \(MFA\) \(#enable-mfa-for-privileged-users\)](#)
- [Update access keys when needed for use cases that require long-term credentials \(#update-access-keys\)](#)
- [Follow best practices to protect your root user credentials \(#lock-away-credentials\)](#)

- Apply least-privilege permissions (#grant-least-privilege)
- Get started with AWS managed policies and move toward least-privilege permissions (#bp-use-aws-defined-policies)
- Use IAM Access Analyzer to generate least-privilege policies based on access activity (#bp-gen-least-privilege-policies)
- Regularly review and remove unused users, roles, permissions, policies, and credentials (#remove-credentials)
- Use conditions in IAM policies to further restrict access (#use-policy-conditions)
- Verify public and cross-account access to resources with IAM Access Analyzer (#bp-preview-access)
- Use IAM Access Analyzer to validate your IAM policies for secure and functional permissions (#best-practice-policy-validation)
- Establish permissions guardrails across multiple accounts (#bp-permissions-guardrails)
- Use permissions boundaries to delegate permissions management within an account (#bp-permissions-boundaries)

## Require human users to use federation with an identity provider to access AWS using temporary credentials

Human users, also known as *human identities*, are the people, administrators, developers, operators, and consumers of your applications. They must have an identity to access your AWS environments and applications. Human users that are members of your organization are also known as *workforce identities*. Human users can also be external users with whom you collaborate, and who interact with your AWS resources. They can do this through a web browser, client application, mobile app, or interactive command-line tools.

Require your human users to use temporary credentials when accessing AWS. You can use an identity provider for your human users to provide federated access to AWS accounts by assuming roles, which provide temporary credentials. For centralized access management, we recommend that you use [AWS IAM Identity Center \(IAM Identity Center\)](https://docs.aws.amazon.com/singlesignon/latest/userguide/getting-started.html) (<https://docs.aws.amazon.com/singlesignon/latest/userguide/getting-started.html>) to manage access to your accounts and permissions within those accounts. You can manage your user identities with IAM Identity Center, or manage access permissions for user identities in IAM Identity Center from an external identity provider. For more

information, see [What is AWS IAM Identity Center](https://docs.aws.amazon.com/singlesignon/latest/userguide/what-is.html) (<https://docs.aws.amazon.com/singlesignon/latest/userguide/what-is.html>) in the *AWS IAM Identity Center User Guide*.

You can use [account access manager](#) ([./account-access-manager.html](#)) — an IAM feature that lets you assign existing IAM roles across your organization's accounts to IAM Identity Center users and groups. Account access manager gives you access to the full IAM role feature set, including custom trust policies, role tags for ABAC, and configurable role paths. You can use it alongside permission sets or on its own.

For more information about roles, see [Roles terms and concepts](#) ([./id\\_roles.html#id\\_roles\\_terms-and-concepts](#)) .

## Require workloads to use temporary credentials with IAM roles to access AWS

A *workload* is a collection of resources and code that delivers business value, such as an application or backend process. Your workload can have applications, operational tools, and components that require credentials to make requests to AWS services, such as requests to read data from Amazon S3.

When you're building on an AWS compute service, such as Amazon EC2 or Lambda, AWS delivers the temporary credentials of an IAM role to that compute resource. Applications written using an AWS SDK will discover and use these temporary credentials to access AWS resources, and there is no need to distribute long lived credentials for an IAM user to your workloads running on AWS.

Workloads that run outside of AWS, such as your on-premises servers, servers from other cloud providers, or managed continuous integration and continuous delivery (CI/CD) platforms, can still use temporary credentials. However, you'll need to deliver these temporary credentials to your workload. The following are ways you can deliver temporary credentials to your workloads:

- You can use IAM Roles Anywhere to request temporary AWS credentials for your workload using an X.509 Certificate from your public key infrastructure (PKI).
- You can call the AWS STS `AssumeRoleWithSAML` API to request temporary AWS credentials for your workload using a SAML assertion from an external identity provider (IdP) that is configured within your AWS account.
- You can call the AWS STS `AssumeRoleWithWebIdentity` API to request temporary AWS credentials for your workload using a JSON web token (JWT) from

an IdP that is configured within your AWS account.

- You can request temporary AWS credentials from your IoT device using Mutual Transport Layer Security (MTLS) authentication using AWS IoT Core.

Some AWS services also support integrations to deliver temporary credentials to your workloads running outside of AWS:

- [Amazon Elastic Container Service \(Amazon ECS\) Anywhere](https://aws.amazon.com/ecs/anywhere/)  (<https://aws.amazon.com/ecs/anywhere/>) lets you run Amazon ECS tasks on your own compute resources, and delivers temporary AWS credentials to your Amazon ECS tasks running on those compute resources.
- [Amazon Elastic Kubernetes Service Hybrid Nodes](https://docs.aws.amazon.com/eks/latest/userguide/hybrid-nodes-overview.html) (<https://docs.aws.amazon.com/eks/latest/userguide/hybrid-nodes-overview.html>) lets you join your compute resources running outside of AWS as nodes to an Amazon EKS cluster. Amazon EKS can deliver temporary credentials to the Amazon EKS pods running on your compute resources.
- [AWS Systems Manager Hybrid Activations](https://docs.aws.amazon.com/systems-manager/latest/userguide/activations.html) (<https://docs.aws.amazon.com/systems-manager/latest/userguide/activations.html>) lets you manage your compute resources running outside of AWS using SSM, and delivers temporary AWS credentials to the SSM agent running on your compute resources.

## Require multi-factor authentication (MFA)

We recommend using IAM roles for human users and workloads that access your AWS resources so that they use temporary credentials. However, for scenarios in which you need an IAM user or root user in your account, require MFA for additional security. With MFA, users have a device that generates a response to an authentication challenge. Each user's credentials and device-generated response are required to complete the sign-in process. For more information, see [AWS Multi-factor authentication in IAM](#) ([./id\\_credentials\\_mfa.html](#)) . We recommend that you use phishing-resistant MFA such as passkeys and security keys wherever possible. For more information, see [Assign a passkey or security key in the AWS Management Console](#) ([./id\\_credentials\\_mfa\\_enable\\_fido.html](#)) .

If you use IAM Identity Center for centralized access management for human users, you can use the IAM Identity Center MFA capabilities when your identity source is configured with the IAM Identity Center identity store, AWS Managed Microsoft AD, or AD Connector. For more information about MFA in IAM Identity Center see [Multi-factor](#)

authentication (<https://docs.aws.amazon.com/singlesignon/latest/userguide/enable-mfa.html>) in the *AWS IAM Identity Center User Guide*.

## Update access keys when needed for use cases that require long-term credentials

Where possible, we recommend relying on temporary credentials instead of creating long-term credentials such as access keys. However, for scenarios in which you need IAM users with programmatic access and long-term credentials, we recommend that you update the access keys when needed, such as when an employee leaves your company. We recommend that you use *IAM access last used information* to update and remove access keys safely. For more information, see [Update access keys \(./id-credentials-access-keys-update.html\)](#).

There are specific use cases that require long-term credentials with IAM users in AWS. Some of the use cases include the following:

- **Workloads that cannot use IAM roles** – You might run a workload from a location that needs to access AWS. In some situations, you can't use IAM roles to provide temporary credentials, such as for WordPress plugins. In these situations, use IAM user long-term access keys for that workload to authenticate to AWS.
- **Third-party AWS clients** – If you are using tools that don't support access with IAM Identity Center, such as third-party AWS clients or vendors that are not hosted on AWS, use IAM user long-term access keys.
- **AWS CodeCommit access** – If you are using CodeCommit to store your code, you can use an IAM user with either SSH keys or service-specific credentials for CodeCommit to authenticate to your repositories. We recommend that you do this in addition to using a user in IAM Identity Center for normal authentication. Users in IAM Identity Center are the people in your workforce who need access to your AWS accounts or to your cloud applications. To give users access to your CodeCommit repositories without configuring IAM users, you can configure the **git-remote-codecommit** utility. For more information about IAM and CodeCommit, see [IAM credentials for CodeCommit: Git credentials, SSH keys, and AWS access keys \(./id\\_credentials\\_ssh-keys.html\)](#). For more information about configuring the **git-remote-codecommit** utility, see [Connecting to AWS CodeCommit repositories with rotating credentials \(https://docs.aws.amazon.com/codecommit/latest/userguide/temporary-access.html#temporary-access-configure-credentials\)](#) in the *AWS CodeCommit User Guide*.

- **Amazon Keyspaces (for Apache Cassandra) access** – In a situation where you are unable to use users in IAM Identity Center, such as for testing purposes for Cassandra compatibility, you can use an IAM user with service-specific credentials to authenticate with Amazon Keyspaces. Users in IAM Identity Center are the people in your workforce who need access to your AWS accounts or to your cloud applications. You can also connect to Amazon Keyspaces using temporary credentials. For more information, see [Using temporary credentials to connect to Amazon Keyspaces using an IAM role and the SigV4 plugin](https://docs.aws.amazon.com/keyspaces/latest/devguide/access.credentials.html#temporary.credentials.IAM) (<https://docs.aws.amazon.com/keyspaces/latest/devguide/access.credentials.html#temporary.credentials.IAM>) in the *Amazon Keyspaces (for Apache Cassandra) Developer Guide*.

## Follow best practices to protect your root user credentials

When you create an AWS account, you establish root user credentials to sign in to the AWS Management Console. Safeguard your root user credentials the same way you would protect other sensitive personal information. To better understand how to secure and scale your root user processes, see [Root user best practices for your AWS account](https://docs.aws.amazon.com/IAM/latest/userguide/root-user-best-practices.html) ([./root-user-best-practices.html](https://docs.aws.amazon.com/IAM/latest/userguide/root-user-best-practices.html)) .

## Apply least-privilege permissions

When you set permissions with IAM policies, grant only the permissions required to perform a task. You do this by defining the actions that can be taken on specific resources under specific conditions, also known as *least-privilege permissions*. You might start with broad permissions while you explore the permissions that are required for your workload or use case. As your use case matures, you can work to reduce the permissions that you grant to work toward least privilege. For more information about using IAM to apply permissions, see [Policies and permissions in AWS Identity and Access Management](https://docs.aws.amazon.com/IAM/latest/userguide/policies.html) ([./access\\_policies.html](https://docs.aws.amazon.com/IAM/latest/userguide/policies.html)) .

## Get started with AWS managed policies and move toward least-privilege permissions

To get started granting permissions to your users and workloads, use the *AWS managed policies* that grant permissions for many common use cases. They are available in your AWS account. Keep in mind that AWS managed policies might not grant least-privilege permissions for your specific use cases because they are available for use by all AWS customers. As a result, we recommend that you reduce permissions further by defining [customer managed policies](#)

([https://docs.aws.amazon.com/IAM/latest/UserGuide/access\\_policies\\_managed-vs-inline.html#customer-managed-policies](https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_managed-vs-inline.html#customer-managed-policies)) that are specific to your use cases. For more information, see [AWS managed policies](#) ([./access\\_policies\\_managed-vs-inline.html#aws-managed-policies](#)) . For more information about AWS managed policies that are designed for specific job functions, see [AWS managed policies for job functions](#) ([./access\\_policies\\_job-functions.html](#)) .

## Use IAM Access Analyzer to generate least-privilege policies based on access activity

To grant only the permissions required to perform a task, you can generate policies based on your access activity that is logged in AWS CloudTrail. [IAM Access Analyzer](#) (<https://docs.aws.amazon.com/IAM/latest/UserGuide/what-is-access-analyzer.html>) analyzes the services and actions that your IAM roles use, and then generates a fine-grained policy that you can use. After you test each generated policy, you can deploy the policy to your production environment. This ensures that you grant only the required permissions to your workloads. For more information about policy generation, see [IAM Access Analyzer policy generation](#) (<https://docs.aws.amazon.com/IAM/latest/UserGuide/access-analyzer-policy-generation.html>) .

## Regularly review and remove unused users, roles, permissions, policies, and credentials

You might have IAM users, roles, permissions, policies, or credentials that you no longer need in your AWS account. IAM provides *last accessed information* to help you identify the users, roles, permissions, policies, and credentials that you no longer need so that you can remove them. This helps you reduce the number of users, roles, permissions, policies, and credentials that you have to monitor. You can also use this information to refine your IAM policies to better adhere to least-privilege permissions. For more information, see [Refine permissions in AWS using last accessed information](#) ([./access\\_policies\\_last-accessed.html](#)) .

## Use conditions in IAM policies to further restrict access

You can specify conditions under which a policy statement is in effect. That way, you can grant access to actions and resources, but only if the access request meets specific conditions. For example, you can write a policy condition to specify that all requests must be sent using TLS. You can also use conditions to grant access to service actions, but only if they are used through a specific AWS service, such as CloudFormation. For more information, see [IAM JSON policy elements: Condition](#) ([./reference\\_policies\\_elements\\_condition.html](#)) .

## Verify public and cross-account access to resources with IAM Access Analyzer

Before you grant permissions for public or cross-account access in AWS, we recommend that you verify if such access is required. You can use IAM Access Analyzer to help you preview and analyze public and cross-account access for supported resource types. You do this by reviewing the [findings](#) (<https://docs.aws.amazon.com/IAM/latest/UserGuide/access-analyzer-findings.html>) that IAM Access Analyzer generates. These findings help you verify that your resource access controls grant the access that you expect. Additionally, as you update public and cross-account permissions, you can verify the effect of your changes before deploying new access controls to your resources. IAM Access Analyzer also monitors supported resource types continuously and generates a finding for resources that allow public or cross-account access. For more information, see [Previewing access with IAM Access Analyzer APIs](#) (<https://docs.aws.amazon.com/IAM/latest/UserGuide/access-analyzer-preview-access-apis.html>) .

## Use IAM Access Analyzer to validate your IAM policies for secure and functional permissions

Validate the policies you create to make sure that they adhere to the [IAM policy language](#) ([./access\\_policies.html#access\\_policies-json](#)) (JSON) and IAM best practices. You can validate your policies by using IAM Access Analyzer policy validation. IAM Access Analyzer provides more than 100 policy checks and actionable recommendations to help you author secure and functional policies. As you author new policies or edit existing policies

in the console, IAM Access Analyzer provides recommendations to help you refine and validate your policies before you save them. Additionally, we recommend that you review and validate all of your existing policies. For more information, see [IAM Access Analyzer policy validation \(https://docs.aws.amazon.com/IAM/latest/UserGuide/access-analyzer-policy-validation.html\)](https://docs.aws.amazon.com/IAM/latest/UserGuide/access-analyzer-policy-validation.html) . For more information about policy checks provided by IAM Access Analyzer, see [IAM Access Analyzer policy check reference \(https://docs.aws.amazon.com/IAM/latest/UserGuide/access-analyzer-reference-policy-checks.html\)](https://docs.aws.amazon.com/IAM/latest/UserGuide/access-analyzer-reference-policy-checks.html) .

## Establish permissions guardrails across multiple accounts

As you scale your workloads, separate them by using multiple accounts that are managed with AWS Organizations. We recommend that you use AWS Organizations [service control policies](https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html)

([https://docs.aws.amazon.com/organizations/latest/userguide/orgs\\_manage\\_policies\\_scps.html](https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html))

(SCPs) to establish permissions guardrails to control access for all principals (IAM roles and users) across your accounts. We recommend that you use AWS Organizations [resource control policies](https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_rcps.html)

([https://docs.aws.amazon.com/organizations/latest/userguide/orgs\\_manage\\_policies\\_rcps.html](https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_rcps.html))

(RCPs) to establish permissions guardrails to control access for AWS resources across your organization. SCPs and RCPs are types of organization policies that you can use to manage permissions in your organization at the AWS organization, organizational unit (OU), or account level.

However, SCPs and RCPs alone are insufficient to grant permissions to principals and resources in your organization. No permissions are granted by SCPs and RCPs. To grant permissions, you must attach [identity-based or resource-based policies](https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_identity-vs-resource.html)

([./access\\_policies\\_identity-vs-resource.html](https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_identity-vs-resource.html)) to IAM users, IAM roles, or the resources in your accounts. For more information, see [SRA building blocks — AWS Organizations, accounts, and guardrails \(https://docs.aws.amazon.com/prescriptive-guidance/latest/security-reference-architecture/organizations.html\)](https://docs.aws.amazon.com/prescriptive-guidance/latest/security-reference-architecture/organizations.html) .

## Use permissions boundaries to delegate permissions management within an account

In some scenarios, you might want to delegate permissions management within an account to others. For example, you could allow developers to create and manage roles for their workloads. When you delegate permissions to others, use *permissions boundaries* to set the maximum permissions that you delegate. A permissions boundary is an advanced feature for using a managed policy to set the maximum permissions that an identity-based policy can grant to an IAM role. A permissions boundary does not grant permissions on its own. For more information, see [Permissions boundaries for IAM entities](#) ([./access\\_policies\\_boundaries.html](#)) .

### Related resources

- [AWS Identity and Access Management API Reference](#)  
(<https://docs.aws.amazon.com/IAM/latest/APIReference/index.html>)
- [AWS CLI commands for AWS Identity and Access Management](#)  
(<https://docs.aws.amazon.com/cli/latest/reference/iam/>)
- [SDKs & Tools](#) [↗](#) (<https://aws.amazon.com/tools/>)

### Get Started

#### Did this page help you?

☒ Yes

☐ No

- [AWS Hands-On Tutorials](#) [↗](#)  
(<https://aws.amazon.com/getting-started/hands-on/>)
- [Provide feedback](#) ([https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden\\_service\\_name=IAM&topic\\_url=https%3A%2F%2Fdocs.aws.amazon.com%2FIAM%2Flatest%2FUserGuide%2Fpractices.html](https://docs.aws.amazon.com/feedback/doc-feedback.html?hidden_service_name=IAM&topic_url=https%3A%2F%2Fdocs.aws.amazon.com%2FIAM%2Flatest%2FUserGuide%2Fpractices.html))
- [AWS Solutions Library](#) [↗](#)  
(<https://aws.amazon.com/solutions/>)
- [AWS Decision Guides](#)  
(<https://docs.aws.amazon.com/decision-guides/>)

### Service Guides

- [Choosing a generative AI service](#)  
(<https://docs.aws.amazon.com/decision-guides/latest/decision-guides/genai-guide.html>)
- [AWS Services](#)  
(<https://docs.aws.amazon.com/#products>)
- [AWS CLI Tutorials on GitHub](#) [↗](#)  
(<https://github.com/aws-samples/sample-developer-tutorials>)



### Developer Tools

- [AWS Code Example Library](#)  
(<https://docs.aws.amazon.com/code-library/latest/ug/what-is-code-library.html>)
- [AWS CLI](#) (<https://docs.aws.amazon.com/cli/>)
- [AWS Builder Center](#) [↗](#)  
(<https://builder.aws.com/>)
- [AWS Developer Tools Blog](#) [↗](#)  
(<https://aws.amazon.com/blogs/developer/>)

### Helpful Links

- [Download the AWS Docs MCP Server](#) [↗](#)  
(<https://github.com/aws-labs/mcp/tree/main/src/aws-documentation-mcp-server>)
- [Sign into the AWS Console](#) [↗](#)  
(<https://console.aws.amazon.com/>)
- [AWS re:Post](#) [↗](#) (<https://repost.aws/>)

English ▼